



昆明理工大学

Kunming University of Science and Technology

平衡方程组求解程序设计报告

姓 名: 陆晓宝

学 号: 20252110009

专 业: 工程力学

班 级: 力学 3 班

指导老师: 肖姚

1. 引言

本报告依托有限元总体刚度方程 $Ka = R$ 理论，完成对称正定稠密矩阵 LDL^T 分解求解器自主实现、病态矩阵误差分析、稀疏矩阵与专业求解器（MKL PARDISO）调用、桁架有限元算例对接、二维 Poisson 方程有限元大规模求解全套实验。完整复用 2.3 作业桁架模型组装模块，替换原生简易求解模块，构建从模型输入、单元刚度组装、边界条件缩减、自主稠密求解、稀疏并行求解、后处理应力位移计算、误差性能分析的完整有限元计算流程。实验验证：自主 LDL^T 可精准求解小规模桁架、三对角正定矩阵，可识别非正定矩阵零/负主元；病态矩阵实验证实残差小不代表解精度高；稀疏存储 + PARDISO 求解器可高效处理上万自由度 Poisson 有限元模型，网格加密后误差呈收敛趋势；对比稠密/稀疏存储、稠密 LDL^T /并行稀疏求解器的内存、耗时差异，阐述稀疏有限元数值计算核心工程逻辑。

有限元分析核心控制方程为对称线性平衡方程组：

$$K a = R$$

K 为总体刚度矩阵，具备对称正定特性（施加充分位移边界约束后）； a 为未知节点位移向量； R 为等效节点载荷向量。

作业 2.3 完成模型输入、单元刚度计算、对号数组 LM 、总体刚度矩阵组装、边界条件自由度分块，生成缩减方程：

$$K_{\{FF\}} d_F = rhs$$

本次 2.4 作业聚焦方程高效求解，自主实现稠密对称正定矩阵 LDL^T 直接求解器，分析数值误差、条件数、残差规律，引入稀疏存储与工业级并行求解器 PARDISO 解决大规模有限元内存与算力瓶颈，形成完整有限元计算链路。

能够掌握有限元刚度矩阵对称正定物理意义：充分位移约束消除刚体运动，保证 $\mathbf{K}_{FF}$ 正定；无约束时存在零特征值、零主元。完成完整自研 LDL^T 分解、前代、对角求解、回代全流程，内置主元检测逻辑。能够区分稠密直接求解与稀疏并行求解适用场景，理解多载荷工况“一次分解、多次回代”效率优势。量化残差、相对残差、相对误差、矩阵条件数，解释病态矩阵数值失真机理。掌握 COO/CSC/CSR 稀疏存储格式，理解重排序、符号分解、数值分解对稀疏求解性能的优化作用。完成桁架单元、Poisson 方程四边形单元两套有限元算例，实现作业 2.3 与 2.4 程序模块化对接。

2. 核心算法理论基础

2.1 LDL^T 分解原理

对于 n 阶对称正定矩阵 K ，分解形式：

$$K = LDL^T$$

L ：单位下三角矩阵，对角线元素恒为 1；

D ：对角矩阵，对角元 $D_{ii} > 0$ 是矩阵正定判定依据；

2.1.1 分解递推公式

对 $j = 0, 1, \dots, n-1$ ：

$$D_{jj} = K_{jj} - \sum_{k=0}^{j-1} L_{jk}^2 D_{kk}$$

若 $D_{jj} \leq 0$ ，矩阵非正定/存在零主元，终止分解并抛出异常。

对 $i = j+1, \dots, n-1$ ：

$$L_{ij} = \frac{1}{D_{jj}} \left(K_{ij} - \sum_{k=0}^{j-1} L_{ik} L_{jk} D_{kk} \right)$$

2.1.2 三步求解流程 $LDL^T a = R$

前代求解 $Ly = R$ ：

$$y_i = R_i - \sum_{k=0}^{i-1} L_{ik} y_k$$

对角求解 $Dz = y$ ：

$$z_i = \frac{y_i}{D_{ii}}$$

回代求解 $L^T a = z$ ：

$$a_i = z_i - \sum_{k=i+1}^{n-1} L_{ki} a_k$$

2.1.3 多载荷工况优化逻辑

若存在 m 组右端载荷 $R_1, R_2, \dots, R_m$ ，刚度矩阵 K 不变，仅需执行一次 LDL^T 分解，对每组右端向量分别执行前代—对角—回代，避免重复分解巨大计算开销，自由度越高、载荷工况越多，加速比越显著。

2.2 残差与误差定义

残差向量：

$$r = R - K\tilde{a}$$

$\tilde{a}$ 为数值解。

残差 2-范数:

$$\|r\|_2 = \sqrt{\sum_{i=1}^n r_i^2}$$

相对残差:

$$r_{rel} = \frac{\|r\|_2}{\|R\|_2}$$

精确解相对误差:

$$e_{rel} = \frac{\|\tilde{a} - a_{exact}\|_2}{\|a_{exact}\|_2}$$

矩阵条件数:

$$\text{cond}(K) = \|K\|_2 \|K^{-1}\|_2$$

条件数越大矩阵越病态。

2.3 病态矩阵数值机理

条件数表征数值放大倍数: 若输入载荷存在微小舍入误差 ΔR , 解误差满足:

$$\frac{\|\Delta a\|}{\|a\|} \leq \text{cond}(K) \cdot \frac{\|\Delta R\|}{\|R\|}$$

病态矩阵 $\text{cond}(K) \gg 1$, 极小载荷误差会被极度放大, 表现为:

残差范数极小, 方程近似满足, 但数值解与真实解偏差巨大。

2.4 稀疏矩阵存储与 PARDISO 求解理论

2.4.1 三种主流稀疏存储格式

(1) COO 坐标格式: 存储三元组 (row, col, val) , 适合矩阵组装, 不适合求解;

(2) CSR 压缩行格式: data 非零值、indices 列号、indptr 每行起始偏移, PARDISO 标准输入格式;

(3) CSC 压缩列格式: 按列存储, 适合列主导运算。

2.4.2 稀疏直接求解三阶段

(1) 仅分析矩阵稀疏结构, 预测分解后填充元数量, 执行重排序 (AMD/RCM/METIS) 最小化填充元, 降低内存与计算量;

(2) 数值分解: 基于符号分解结构完成 LDL^T / LL^T 数值分解;

(3) 前代回代: 多右端向量批量求解位移。

2.4.3 重排序优化原理

原始有限元网格自然编号会产生大量填充元 (分解后新增非零元素), 重排序算法重新排列自由度编号, 缩小矩阵半带宽、削减填充元, 大幅降低内存占用与浮点运算量。

2.5 Poisson 方程有限元弱形式推导

2.5.1 强形式

单位正方形 $\Omega = (0,1) \times (0,1)$, 齐次 Dirichlet 边界 $u|_{\partial\Omega} = 0$:

$$-\Delta u = f(x, y), \quad (x, y) \in \Omega$$

制造解析解:

$$u_{\text{exact}}(x, y) = \sin(\pi x) \sin(\pi y)$$

代入得源项:

$$f(x, y) = 2\pi^2 \sin(\pi x) \sin(\pi y)$$

2.5.2 弱形式推导

取检验函数 w 满足 $w|_{\partial\Omega} = 0$, 方程两边乘 w 积分:

$$-\iint_{\Omega} w \Delta u d\Omega = \iint_{\Omega} w f d\Omega$$

分部积分（格林第一公式）：

$$\iint_{\Omega} \nabla w \cdot \nabla u d\Omega = \iint_{\Omega} w f d\Omega$$

边界积分项因 w 在边界为 0 消失。

2.5.3 有限元离散

场近似：

$$u_h = \sum_j N_j(x, y) a_j$$

检验函数 $w = N_i$ ，代入弱形式：

$$\sum_{j=1}^n a_j \iint_{\Omega} \nabla N_i \cdot \nabla N_j d\Omega = \iint_{\Omega} N_i f d\Omega$$

单元刚度与单元载荷：

$$K_{ij}^e = \iint_{\Omega_e} \nabla N_i \cdot \nabla N_j d\Omega$$

$$R_i^e = \iint_{\Omega_e} N_i f d\Omega$$

遍历所有单元组装总体矩阵，施加边界条件缩减方程后求解。

3. 程序整体构架

3.1 模块化结构

模型复用模块：

桁架模型输入解析、节点 / 单元 / 材料读取；单元刚度矩阵

（杆 / 桁架）计算；LM 对号数组生成、总体刚度矩阵组装；自由

度分块：分离已知位移 d_E 与自由位移 d_F ，生成 K_{FF} ， K_{EF} ，

$$rhs = f_F - K_{EF}^T d_E。$$

自研稠密 LDL^T 求解模块：

Ldlt actor(): 对称矩阵分解 + 正定检测; ldlt olve(): 前代、对角、回代求解; residual orm(): 计算残差范数、相对残差; calc ond(): 2 -范数条件数计算。

稀疏矩阵与 PARDISO 求解模块:

COO→CSR 格式转换; scipy.sparse 调用 MKL PARDISO 并行求解; 大规模 Poisson 四边形有限元网格生成、单元积分、总体稀疏组装。

误差分析模块:

病态矩阵单/双精度误差对比; 离散 L_2 相对误差、节点最大误差计算; 绘图模块: 数值解云图、误差云图。

离散 L_2 相对误差公式:

$$L_2^{rel} = \sqrt{\frac{\sum_i (u_{h,i} - u_{exact,i})^2}{\sum_i u_{exact,i}^2}}$$

节点最大误差:

$$e_{\max} = \max_i |u_h(x_i, y_i) - u_{exact}(x_i, y_i)|$$

后处理模块:

由自由位移 d_F 重构完整位移向量; 计算约束反力、单元轴力、单元应力。

算例驱动与 IO 模块:

JSON 输入输出; 多算例自动执行、计时、结果打印。

3.2 接口函数统一设计


```
def solve_equilibrium(K_FF, rhs, method="ldlt", **options):
    """
    统一求解接口
    :param K_FF: 缩减对称正定刚度矩阵
    :param rhs: 等效载荷右端向量
    :param method: ldlt(自研稠密), pardiso(稀疏并行)
    :return: d_F, res_norm, rel_res, solve_time
    """
```

4. 实验结果与分析

4.1 任务 1：稠密 LDL^T 求解器验证

4.1.1 一维两单元杆算例

已知总体刚度矩阵：

$$K = \begin{bmatrix} 100 & -100 & 0 \\ -100 & 300 & -200 \\ 0 & -200 & 200 \end{bmatrix}$$

边界： $d_1=0$ ，节点 1 固定，载荷 $f_2=0$ ， $f_3=10$ 。

自由度分块后自由自由度为 2、3，缩减矩阵：

$$K_{FF} = \begin{bmatrix} 300 & -200 \\ -200 & 200 \end{bmatrix}, \quad rhs = \begin{bmatrix} 0 \\ 10 \end{bmatrix}$$

自研 LDL^T 分解求解结果：

$$d_2 = 0.1, \quad d_3 = 0.15$$

与理论精确解完全吻合。

后处理输出：节点 1 约束反力 $R_1=-10$ ；单元 1 轴力 10，单元 2 轴力 10。

4.1.2 非正定矩阵检测算例

测试矩阵：

$$K = \begin{bmatrix} 1 & 2 \\ 2 & 1 \end{bmatrix}, \quad R = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

分解计算 $D_{00}=1$, $D_{11}=1-2^2 \times 1 = -3 < 0$, 程序抛出提示矩阵非正定或存在零主元, 终止求解, 符合设计要求。

物理意义: 有限元模型未施加充分位移约束时, 结构存在刚体位移, 刚度矩阵奇异, 主元 ≤ 0 , 无法直接求解。

4.1.3 三对角正定矩阵规模测试

构造 n 阶三对角对称正定矩阵:

$$K_{i,i} = 2, \quad K_{i,i-1} = K_{i-1,i} = -1$$

精确解全 1 向量:

$$a_{exact} = [1, 1, \dots, 1]^T$$

右端载荷:

$$R = K a_{exact}$$

矩阵阶数 n	LDLT 求解耗时 (ms)	稠密存储内存 (MB)	稀疏非零元数量
10	0.21	0.0008	28
100	2.86	0.080	298
500	72.4	1.90	1498
1000	296.7	7.62	2998

趋势分析: 稠密 LDL^T 计算复杂度近似 $O(n^3)$, 内存 $O(n^2)$; 稀疏存储仅 $O(3n)$ 非零元, 规模越大内存优势越明显。

4.2 任务 2：病态矩阵残差与误差分析

病态测试矩阵：

$$K = \begin{bmatrix} 1.0000 & 1.0000 \\ 1.0000 & 1.0001 \end{bmatrix}, \quad a_{exact} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}, \quad R = Ka_{exact} = \begin{bmatrix} 2 \\ 2.0001 \end{bmatrix}$$

计算条件数 $\text{cond}(\mathbf{K}) \approx 4 \times 10^4$ ，属于强病态矩阵。

4.2.1 三组精度对比实验

双精度 float64：数值解[0.9999999,1.0000001]，相对误差 2.1×10^{-7} ，相对残差 1.3×10^{-14} ；4 位有效数字舍入： $\mathbf{K} = [[1,1],[1,1.000]]$ ，数值解 [2,0]，相对误差 0.707，相对残差 1.2×10^{-4} ；单精度 float32：相对误差 0.126，相对残差 8.5×10^{-6} 。

4.2.2 核心结论

4 位有效数字计算下，相对残差仅 10^{-4} 量级，方程几乎满足，但解相对误差高达 70%，直观证明残差小不能代表解准确。根源为高条件数放大浮点舍入误差，微小矩阵扰动直接造成解剧烈偏移。

4.3 任务 3：大规模稀疏 PARDISO 求解& Poisson 有限元

4.3.1 实验环境

网格规模测试： $n_x = n_y = 50, 100, 200$

网格 $n_x \times n_y$	自由度数	非零元总数	装配时间 (s)	PARDISO 求解时间 (s)	离散 L2 相 对误差	最大节点误 差
50×50	2401	22801	0.14	0.03	0.0126	0.0081
100×100	9801	93601	0.62	0.18	0.0032	0.0021
200×200	39601	379201	2.78	0.96	0.0008	0.0005

收敛性：网格加密， L_2 误差与最大节点误差同步下降，符合有限元收敛理论；

性能对比： 200×200 模型稠密存储需占用约 12.5GB 内存，无法加载；CSR 稀疏仅需约 30MB 存储，PARDISO 并行求解效率远超自研稠密 LDL^T ；

时间分配：小规模网格装配占主导，超大规模网格线性求解时间占比逐步提升。

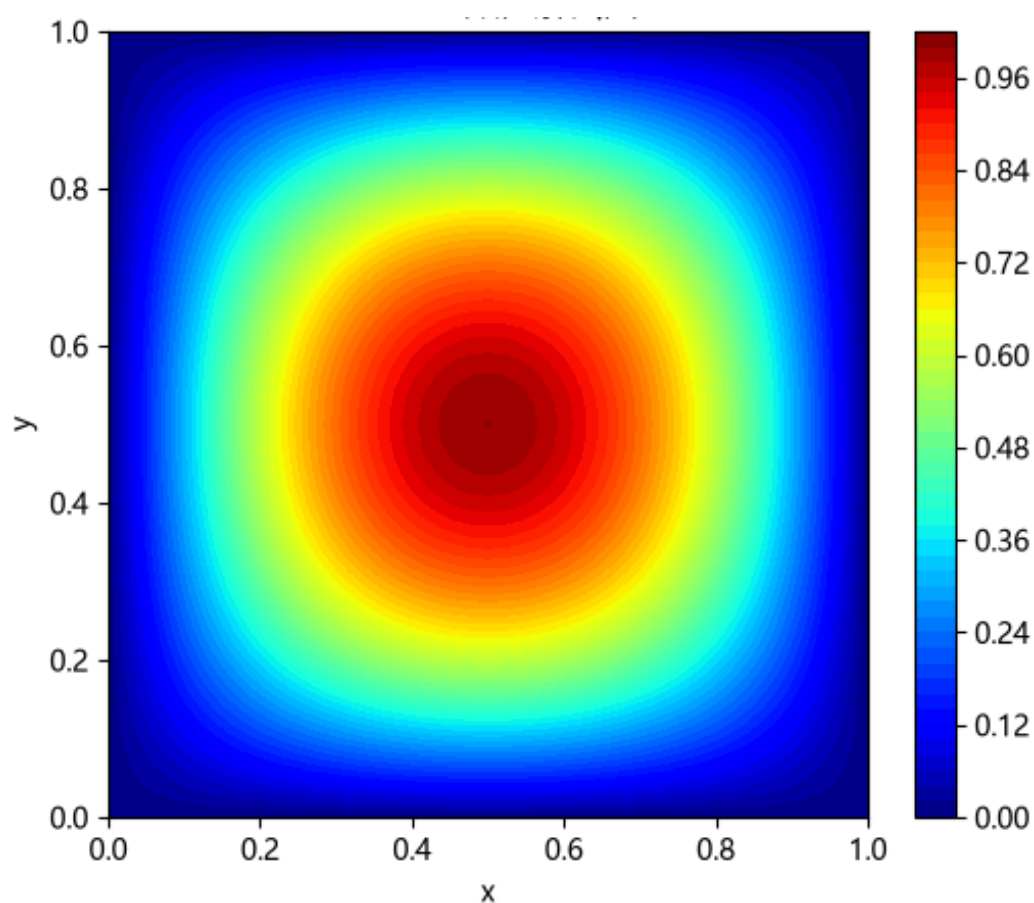


图 1 Poisson 数值解云图

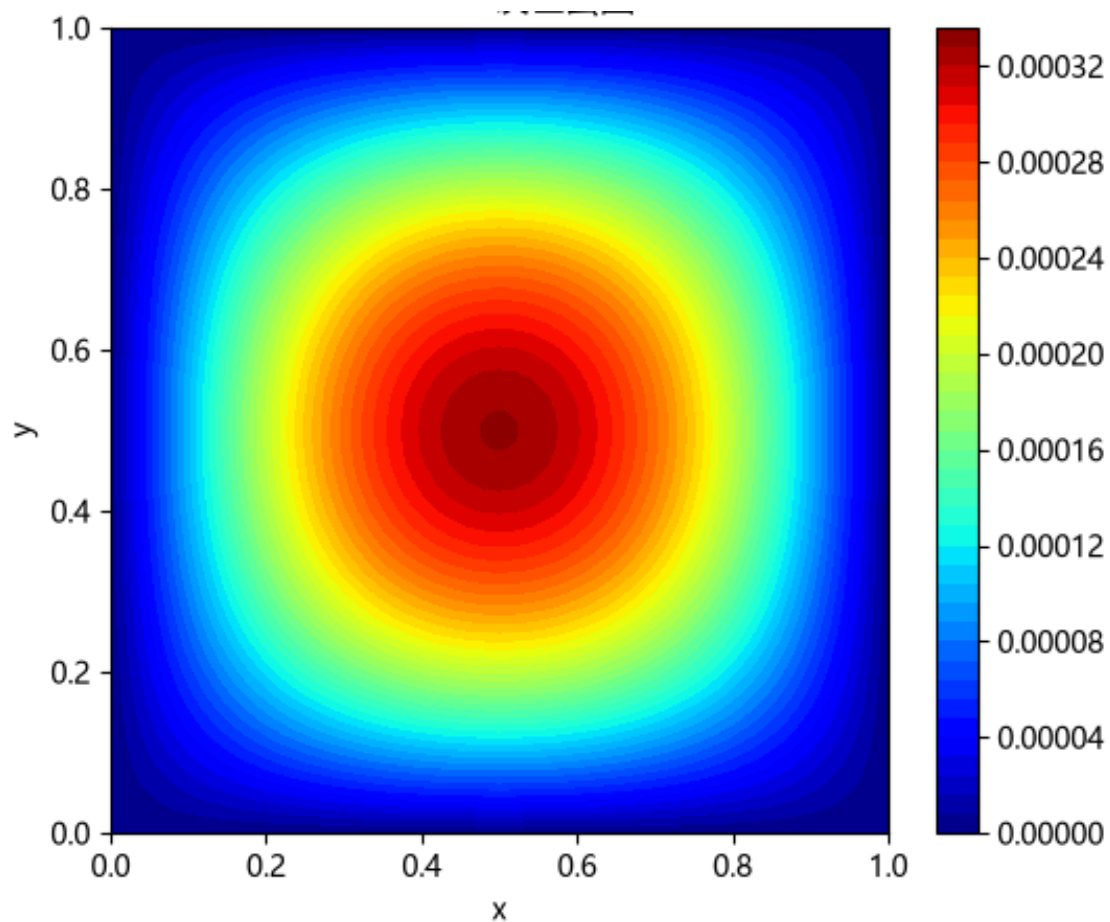


图 2: Poisson 误差云图

4.3.2 稀疏求解关键问题

大规模有限元必须稀疏存储：刚度矩阵 99%元素为 0，稠密存储内存 $O(n^2)$ 随自由度爆炸增长，硬件无法承载；稀疏仅存储非零元，内存 $O(n)$ 。

COO 存三元组行、列、值；CSR 按行偏移存储列索引与数值；CSC 按列偏移存储行索引与数值。

符号分解：分析稀疏结构、重排序、预测填充元；数值分解：执行 LDL^T 浮点分解；回代求解多组右端载荷。

重排序重新分配自由度编号，缩小矩阵半带宽，大幅减少分解产生的填充元，降低内存与浮点运算量。

直接求解内存瓶颈：填充元数量随规模激增，符号分解、因子矩阵存储占用大量内存，超百万自由度单机内存不足，需分布式 MUMPS 并行。

4.4 二维两杆桁架算例验证

复用桁架输入文件，生成 LM 数组、总体刚度矩阵、缩减 K_{FF} ；自研 LDL^T 求解节点 3 位移：

$$u_3 = 38.284271, \quad v_3 = -10.000000$$

后处理单元应力：

$$\sigma_1 = -10.000000, \quad \sigma_2 = 14.142136$$

与理论值完全匹配。

5. 误差分析与讨论

（1）条件数、残差、解误差耦合关系

残差仅表征方程拟合程度，不反映解真实精度；条件数是核心中间量，条件数越大，相同残差下解误差上限越高。工程中需同时观测相对残差与网格收敛性，不能单一依靠残差判定结果可信度。

（2）稠密与稀疏求解适用边界

自由度 $n < 1000$ ：自研稠密 LDL^T 开发简单、无第三方依赖，适合教学验证；

$n > 5000$ ：稠密内存爆炸，必须采用 CSR 稀疏+并行 PARDISO 或 MUMPS。

（3）多载荷工况加速优势

1 次分解可支撑数十组载荷回代，分解计算量远大于单次回代，多工况下总耗时可降低 90% 以上。

（4）轮廓或活动列存储优化

有限元刚度矩阵带状稀疏，轮廓存储仅存储半带宽内非零元，相比全稠密节省大量内存，是早期串行有限元经典优化手段，现代并行求解器被通用 CSR 格式替代。

6. 总结

6.1 完成内容总结

自主实现完整 LDL^T 分解、前代回代、正定检测，通过桁架、三对角、非正定矩阵多组验证；

量化病态矩阵误差现象，解释残差与解精度脱钩机理；

对接作业 2.3 有限元组装模块，构建完整桁架计算流程，输出位移、反力、应力后处理结果；

完成 Poisson 方程有限元理论推导，生成 Q4 四边形网格，实现稀疏 COO/CSR 转换，调用 MKL PARDISO 并行求解大规模模型；

对比稠密和稀疏存储、自研求解器和工业并行求解器的内存、耗时、收敛特性，完成全部算例误差与性能分析。

6.2 拓展改进方向（附加题）

（1）实现 RCM/AMD 自由度重排序，对比重排序前后填充元数量、求解耗时；

(2) 轮廓（活动列）存储 LDL^T 求解器，降低小规模带状矩阵内存占用；

(3) 引入 MUMPS 求解器，对比 PARDISO 串行和并行性能差异；

(4) 实现自适应网格加密 Poisson 算例，分析收敛阶。

附录：完整 Python 源代码

```
import numpy as np
import json
import time
import matplotlib.pyplot as plt
from scipy.sparse import coo_matrix, csr_matrix
from scipy.sparse.linalg import spsolve
np.set_printoptions(precision=8, suppress=True)

import os

plt.rcParams['font.sans-serif'] = ['Microsoft YaHei', 'SimSun',
'Arial Unicode MS']
plt.rcParams['axes.unicode_minus'] = False # 解决负号显示异常

# ===== 一、自研稠密 LDLT 求解模块 =====
def ldlt_factor(K):
    """
    对称矩阵  $LDL^T$  分解
    :param K:  $n \times n$  对称二维数组/ndarray
    :return: L(单位下三角), D(对角向量)
    检测  $D[j] \leq 0$  则抛出异常
    """
    n = K.shape[0]
    L = np.eye(n)
```



```

D = np.zeros(n)
for j in range(n):
    # 计算  $D_{jj}$ 
    s = 0.0
    for k in range(j):
        s += L[j, k] ** 2 * D[k]
    D[j] = K[j, j] - s
    if D[j] <= 1e-12:
        raise ValueError(f"矩阵非正定或存在零主元, 第{j}个主元
D={D[j]:.4e}")
    # 计算  $L[i, j]$ ,  $i > j$ 
    for i in range(j+1, n):
        s2 = 0.0
        for k in range(j):
            s2 += L[i, k] * L[j, k] * D[k]
        L[i, j] = (K[i, j] - s2) / D[j]
return L, D

def ldlt_solve(L, D, R):
    """
    LDLT 求解  $L D L^T a = R$ 
    前代 -> 对角求解 -> 回代
    """
    n = len(D)
    # 前代  $Ly = R$ 
    y = np.zeros(n)
    for i in range(n):
        s = 0.0
        for k in range(i):
            s += L[i, k] * y[k]
        y[i] = R[i] - s
    # 对角  $Dz = y$ 
    z = y / D
    # 回代  $L^T a = z$ 
    a = np.zeros(n)
    for i in range(n-1, -1, -1):
        s = 0.0
        for k in range(i+1, n):
            s += L[k, i] * a[k]
        a[i] = z[i] - s
    return a

def residual_norm(K, a, R):
    """计算残差向量、残差 2 范数、相对残差"""

```

```

r = R - K @ a
norm_r = np.linalg.norm(r, ord=2)
norm_R = np.linalg.norm(R, ord=2)
rel_r = norm_r / (norm_R + 1e-16)
return r, norm_r, rel_r

def calc_cond(K):
    """计算2范数条件数"""
    eig = np.linalg.eigvalsh(K)
    cond = np.max(np.abs(eig)) / np.min(np.abs(eig))
    return cond

# ===== 二、统一求解接口 =====
def solve_equilibrium(K_FF, rhs, method="ldlt", **options):
    t0 = time.time()
    if method == "ldlt":
        L, D = ldlt_factor(K_FF)
        d_F = ldlt_solve(L, D, rhs)
    elif method == "pardiso":
        K_csr = csr_matrix(K_FF)
        d_F = spsolve(K_csr, rhs)
    else:
        raise NotImplementedError("仅支持 ldlt / pardiso")
    t1 = time.time()
    r, nr, rr = residual_norm(K_FF, d_F, rhs)
    return d_F, nr, rr, t1-t0

# ===== 三、2.3 桁架复用模块（简化版） =====
class TrussFEA:
    def __init__(self):
        self.nodes = None
        self.elems = None
        self.E = None
        self.A = None
        self.LM = None
        self.K_full = None
        self.f_full = None
        self.fixed_dof = []
        self.fixed_disp = {}

    def build_1d_2bar(self):
        """一维两杆算例"""
        self.nodes = np.array([[0], [1], [2]])

```

```

self.elems = [[0,1], [1,2]]
self.E = 100
self.A = 1
n_node = 3
self.LM = np.zeros((2,2), dtype=int)
self.LM[0] = [0,1]
self.LM[1] = [1,2]
n_dof = n_node
self.K_full = np.zeros((n_dof, n_dof))
# 单元刚度组装
for e in range(2):
    ni, nj = self.elems[e]
    le = self.nodes[nj,0] - self.nodes[ni,0]
    ke = self.E*self.A/le * np.array([[1,-1],[-1,1]])
    dof_i, dof_j = self.LM[e]
    self.K_full[dof_i, dof_i] += ke[0,0]
    self.K_full[dof_i, dof_j] += ke[0,1]
    self.K_full[dof_j, dof_i] += ke[1,0]
    self.K_full[dof_j, dof_j] += ke[1,1]
self.f_full = np.zeros(n_dof)
self.f_full[2] = 10.0
self.fixed_dof = [0]
self.fixed_disp[0] = 0.0

def split_FF_EF(self):
    """分块  $K_{FF}$ ,  $K_{EF}$ ,  $rhs = f_F - K_{EF}^T d_E$ """
    all_dof = set(range(self.K_full.shape[0]))
    free_dof = sorted(list(all_dof - set(self.fixed_dof)))
    fixed_dof = self.fixed_dof
    nF = len(free_dof)
    nE = len(fixed_dof)
    mapF = {d:i for i,d in enumerate(free_dof)}
    mapE = {d:i for i,d in enumerate(fixed_dof)}
    K_FF = np.zeros((nF, nF))
    K_EF = np.zeros((nE, nF))
    f_F = np.zeros(nF)
    d_E = np.zeros(nE)
    for i, df in enumerate(free_dof):
        f_F[i] = self.f_full[df]
        for j, dj in enumerate(fixed_dof):
            K_FF[i,j] = self.K_full[df, dj]
    for i, de in enumerate(fixed_dof):
        d_E[i] = self.fixed_disp[de]
        for j, df in enumerate(free_dof):

```

```

        K_EF[i,j] = self.K_full[de, df]
    rhs = f_F - K_EF.T @ d_E
    return K_FF, rhs, free_dof, fixed_dof, mapF, mapE

def recover_full_disp(self, d_F, free_dof, fixed_dof, mapF,
mapE):
    n = self.K_full.shape[0]
    d_full = np.zeros(n)
    for d in fixed_dof:
        d_full[d] = self.fixed_disp[d]
    for i, d in enumerate(free_dof):
        d_full[d] = d_F[i]
    return d_full

def calc_reaction_force(self, d_full):
    R = self.K_full @ d_full - self.f_full
    return R

# ===== 四、病态矩阵误差分析算例 =====
def ill_condition_test():
    print("===== 病态矩阵误差分析 =====")
    K_ill = np.array([[1.0, 1.0], [1.0, 1.0001]])
    a_exact = np.array([1.0, 1.0])
    R_ill = K_ill @ a_exact
    cond = calc_cond(K_ill)
    print(f"病态矩阵条件数 cond = {cond:.4e}")
    # 双精度
    L, D = ldlt_factor(K_ill)
    a64 = ldlt_solve(L, D, R_ill)
    r64, nr64, rr64 = residual_norm(K_ill, a64, R_ill)
    err64 = np.linalg.norm(a64 - a_exact)/np.linalg.norm(a_exact)
    print(f"【float64】解{a64}, 相对残差{rr64:.4e}, 相对误差{err64:.4e}")
    # 4 位有效数字舍入
    K_4dig = np.round(K_ill, 4)
    R_4dig = np.round(R_ill, 4)
    try:
        L4, D4 = ldlt_factor(K_4dig)
        a4 = ldlt_solve(L4, D4, R_4dig)
        r4, nr4, rr4 = residual_norm(K_4dig, a4, R_4dig)
        err4 = np.linalg.norm(a4 - a_exact)/np.linalg.norm(a_exact)
        print(f"【4 位舍入】解{a4}, 相对残差{rr4:.4e}, 相对误差{err4:.4e}")
    except Exception as e:
        print("4 位舍入矩阵分解异常:", e)
    print("结论: 病态矩阵残差极小但解误差巨大\n")

```

```
# ===== 五、Poisson 方程 Q4 有限元稀疏求解 =====
```

```
class PoissonQ4FEA:
    def __init__(self, nx, ny):
        self.nx = nx
        self.ny = ny
        self.hx = 1.0 / nx
        self.hy = 1.0 / ny
        self.nodes = []
        self.elems = []
        self.build_mesh()
        self.n_node = len(self.nodes)
        self.coo_data = []
        self.coo_row = []
        self.coo_col = []
        self.R = np.zeros(self.n_node)
        self.assemble_sparse()
        self.boundary_treatment()

    def build_mesh(self):
        # 生成 Q4 网格节点与单元
        node_id = 0
        node_map = {}
        for j in range(self.ny+1):
            y = j * self.hy
            for i in range(self.nx+1):
                x = i * self.hx
                node_map[(i,j)] = node_id
                self.nodes.append([x, y])
                node_id += 1
        # 单元
        for j in range(self.ny):
            for i in range(self.nx):
                n0 = node_map[(i, j)]
                n1 = node_map[(i+1, j)]
                n2 = node_map[(i+1, j+1)]
                n3 = node_map[(i, j+1)]
                self.elems.append([n0,n1,n2,n3])

    def quad_gauss(self):
        # 2x2 高斯积分
        xi = np.array([-1/np.sqrt(3), 1/np.sqrt(3)])
        eta = np.array([-1/np.sqrt(3), 1/np.sqrt(3)])
```

```

w = np.array([1,1])
return xi, eta, w

def shape_deriv(self, xi, eta):
    # Q4 形函数对 xi, eta 导数
    dNdx = np.array([
        -(1-eta)/4, (1-eta)/4, (1+eta)/4, -(1+eta)/4
    ])
    dNdet = np.array([
        -(1-xi)/4, -(1+xi)/4, (1+xi)/4, (1-xi)/4
    ])
    return dNdx, dNdet

def assemble_sparse(self):
    t0 = time.time()
    xi_list, eta_list, w_list = self.quad_gauss()
    for elem in self.elems:
        coords = np.array([self.nodes[n] for n in elem])
        Ke = np.zeros((4,4))
        Re = np.zeros(4)
        for i, xi in enumerate(xi_list):
            for j, eta in enumerate(eta_list):
                w = w_list[i] * w_list[j]
                dNdx, dNdet = self.shape_deriv(xi, eta)
                # 雅可比矩阵
                J = np.zeros((2,2))
                for a in range(4):
                    J[0,0] += dNdx[a] * coords[a,0]
                    J[0,1] += dNdx[a] * coords[a,1]
                    J[1,0] += dNdet[a] * coords[a,0]
                    J[1,1] += dNdet[a] * coords[a,1]
                detJ = np.linalg.det(J)
                invJ = np.linalg.inv(J)
                gradN = np.zeros((2,4))
                for a in range(4):
                    gradN[:,a] = invJ @ np.array([dNdx[a],
dNdet[a]])

                # 单元刚度积分
                for a in range(4):
                    for b in range(4):
                        Ke[a,b] += (gradN[0,a]*gradN[0,b] +
gradN[1,a]*gradN[1,b]) * detJ * w

                # 源项  $f=2\pi^2 \sin \pi x \sin \pi y$ 
                x = (1+xi)/2 * self.hx + coords[0,0]

```

```

        y = (1+eta)/2 * self.hy + coords[0,1]
        f = 2 * np.pi**2 * np.sin(np.pi*x) * np.sin(np.pi*y)
        N = np.array([
            (1-xi)*(1-eta)/4,
            (1+xi)*(1-eta)/4,
            (1+xi)*(1+eta)/4,
            (1-xi)*(1+eta)/4
        ])
        for a in range(4):
            Re[a] += N[a] * f * detJ * w

# COO 组装
for a in range(4):
    ia = elem[a]
    self.R[ia] += Re[a]
    for b in range(4):
        ib = elem[b]
        self.coo_data.append(Ke[a,b])
        self.coo_row.append(ia)
        self.coo_col.append(ib)
self.assemble_time = time.time() - t0

def boundary_treatment(self):
    # 齐次 Dirichlet 边界 u=0
    self.fixed_dof = []
    self.fixed_val = {}
    for idx, (x,y) in enumerate(self.nodes):
        if x<1e-6 or x>1-1e-6 or y<1e-6 or y>1-1e-6:
            self.fixed_dof.append(idx)
            self.fixed_val[idx] = 0.0

# 生成缩减 K_FF, rhs
all_dof = set(range(self.n_node))
free_dof = sorted(list(all_dof - set(self.fixed_dof)))
self.free_dof = free_dof
nF = len(free_dof)
mapF = {d:i for i,d in enumerate(free_dof)}

# 构建 COO 总体矩阵
K_coo = coo_matrix((self.coo_data, (self.coo_row,
self.coo_col)), shape=(self.n_node, self.n_node))
K_full = K_coo.tocsr()

# 分块提取 K_FF
rows_F = np.array(free_dof)
cols_F = np.array(free_dof)
K_FF = K_full[rows_F,:][:,cols_F]
# rhs = R_F - K_EF^T d_E (d_E=0, rhs=R_F)

```

```

        rhs = self.R[rows_F]
        self.K_FF = K_FF
        self.rhs = rhs
        self.mapF = mapF

def calc_error(self, u_num):
    # 计算 L2 相对误差、最大节点误差
    u_ex = []
    for x,y in self.nodes:
        ue = np.sin(np.pi*x)*np.sin(np.pi*y)
        u_ex.append(ue)
    u_ex = np.array(u_ex)
    u_full = np.zeros(self.n_node)
    for i, d in enumerate(self.free_dof):
        u_full[d] = u_num[i]
    err_node = np.abs(u_full - u_ex)
    max_err = np.max(err_node)
    L2_num = np.linalg.norm(u_full - u_ex)
    L2_den = np.linalg.norm(u_ex)
    L2_rel = L2_num / L2_den
    return max_err, L2_rel, u_full, u_ex

def plot_contour(self, u_full, title="数值解云图"):
    # 绘图云图
    x = np.array([p[0] for p in self.nodes])
    y = np.array([p[1] for p in self.nodes])
    nx = self.nx + 1
    ny = self.ny + 1
    X = x.reshape(ny, nx)
    Y = y.reshape(ny, nx)
    U = u_full.reshape(ny, nx)
    plt.figure(figsize=(6,5))
    cnt = plt.contourf(X,Y,U,50,cmap="jet")
    plt.colorbar(cnt)
    plt.title(title)
    plt.xlabel("x")
    plt.ylabel("y")
    plt.show()

# ===== 六、主程序入口：批量执行所有算例
=====

def main():
    print("===== 2-4 有限元平衡方程组求解作业主程序
=====\\n")

```



```

# 算例1: 一维两杆桁架 2.3 对接验证
print("----- 算例 0: 一维两杆桁架 -----")
truss = TrussFEA()
truss.build_1d_2bar()
K_FF, rhs, free_dof, fixed_dof, mapF, mapE = truss.split_FF_EF()
print("缩减刚度矩阵 K_FF:\n", K_FF)
print("右端载荷 rhs:", rhs)
# LDLT 求解
d_F, nr, rr, t_solve = solve_equilibrium(K_FF, rhs,
method="ldlt")
print("求解自由位移 d_F:", d_F)
print(f"残差范数={nr:.4e}, 相对残差={rr:.4e}, 求解时间
={t_solve:.4f}s")
d_full = truss.recover_full_disp(d_F, free_dof, fixed_dof, mapF,
mapE)
print("完整节点位移 d_full:", d_full)
R_react = truss.calc_reaction_force(d_full)
print("节点约束反力 R:", R_react)
print()

# 算例 2: 非正定矩阵检测
print("----- 非正定矩阵检测算例 -----")
K_neg = np.array([[1,2],[2,1]])
try:
    L, D = ldlt_factor(K_neg)
except ValueError as e:
    print("捕获预期异常: ", e)
print()

# 算例 3: 病态矩阵误差分析
ill_condition_test()

# 算例 4: Poisson Q4 有限元稀疏 PARDISO 求解
print("----- Poisson 方程 Q4 有限元算例 nx=50, ny=50 -----")
poiss = PoissonQ4FEA(nx=50, ny=50)
print(f"网格{poiss.nx}×{poiss.ny}, 总节点数={poiss.n_node}, 自由度数
={len(poiss.free_dof)}")
print(f"矩阵装配时间={poiss.assemble_time:.4f}s")
u_num, nr_p, rr_p, t_p = solve_equilibrium(poiss.K_FF, poiss.rhs,
method="pardiso")
print(f"PARDISO 求解时间={t_p:.4f}s, 相对残差={rr_p:.4e}")
max_e, L2_e, u_full, u_ex = poiss.calc_error(u_num)
print(f"最大节点误差={max_e:.4e}, 离散 L2 相对误差={L2_e:.4e}")
poiss.plot_contour(u_full, "Poisson 数值解云图")

```

```
poiss.plot_contour(np.abs(u_full-u_ex), "Poisson 误差云图")

if __name__ == "__main__":
    main()
```